

Veuillez m'excuser pour l'oubli des dépôts atomiques. Les dépôts sont mis questions par questions, avec les changements numérotés.

- 1) Le micro-service http vient d'être créé. Je précise qu'il faut un http et le port, 1234. On y remarque le code `<h1>Hello Word</h1>`.
Pour mettre le service en route, j'utilise donc la commande `node question1.js`

2)

```
C:\Users\florent>curl http://localhost:1234/  
<h1>Hello world !</h1>  
C:\Users\florent>
```

- 3) Pour apercevoir l'URL demandée, il suffit d'ajouter au programme :
`console.log(` URL demandée : ${req.url}`);`

Ainsi, quand je lance le serveur et que je me connecte côté client, j'ai bien l'URL qui apparaît (ici la racine) :

```
S.  
C:\Users\florent>curl http://localhost:1234/  
<h1>Hello world !</h1>  
C:\Users\florent>curl http://localhost:1234/  
<h1>Hello world !</h1>  
C:\Users\florent>curl http://localhost:1234/  
<h1>Hello world !</h1>  
C:\Users\florent>  
flore@LAPTOP-90B3947M UCRT64 ~/Desktop/R&T 2m  
1_Florent_REUS-DORANDEU (main)  
$ node Question3.js  
Serveur en écoute sur http://localhost:1234  
URL demandée : /  
URL demandée : /  
URL demandée : /  
URL demandée : /  
URL demandée : /  
URL demandée : /  
URL demandée : /  
URL demandée : /
```

4)

```
flore@LAPTOP-90B3947M UCRT64 ~/Desktop/R&T 2
1_Florent_REUS-DORANDEU (main)
$ node Question4.js
Serveur en écoute sur http://localhost:1234
Client ::ffff:127.0.0.1:65427 -> /
Client ::ffff:127.0.0.1:65428 -> /
Client ::ffff:127.0.0.1:65428 -> /
Client ::ffff:127.0.0.1:65428 -> /
Client ::1:65434 -> /
Client ::1:65435 -> /
Client ::ffff:127.0.0.1:65436 -> /
Client ::1:65453 -> /oui
```

On remarque bien l'adresse IP de mon ordinateur 127.0.0.1 (normal, car hébergé en local) ainsi que le port utilisé.

On remarque l'adresse IPV4 loopback de ma machine quand je tape l'URL sur un navigateur, et l'adresse IPV6 loopback après un curl, ainsi que les différents ports utilisés.

Je remarque qu'après la commande curl -4 <http://localhost:1234/oui>, j'ai le retour de l'adresse IPv4. `Client ::ffff:127.0.0.1:61124 -> /oui`

- 5) Je récupère l'URL entrée par le client, puis la compare avec la racine /. C'est les 2 sont /, alors la page n'indique aucune erreur, si il y a différence, alors on a erreur 404.

```
C:\Users\flore>curl -4 http://localhost:1234/oui
<h1>404 Not Found</h1>
C:\Users\flore>curl -4 http://localhost:1234/
<h1>Hello world !</h1>
```

- 6)

```

if (racine === '/') {
  res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
  res.end('<h1>Hello world !</h1>');
} else {
  // "/ok.html" -> lecture du fichier fichiersq6/ok.html
  fs.readFile(__dirname + '/fichiersq6' + racine, (err, data) => {
    if (!err) {
      res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
      res.end(data);
    } else if (err.code === 'EACCES' || err.code === 'EPERM') {
      // EACCES pour Linux ou EPERM pour Windows : pas le droit de lire
      res.writeHead(403, { 'Content-Type': 'text/html; charset=utf-8' });
      res.end('<h1>403 Forbidden</h1>');
    } else {
      res.writeHead(404, { 'Content-Type': 'text/html; charset=utf-8' });
      res.end('<h1>404 Not Found</h1>');
    }
  });
}
}

```

On remplit le répertoire existant avec le répertoire actuel (__dirname), le dossier où est stocké les fichiers accessibles (fichiersq6) plus l'URL (racine). Si l'URL correspond à un fichier déjà existant (et si les droits sont bons), il affiche le fichier. Si les droits ne sont pas bons, il affiche 403 Forbidden (EACCES est le code d'erreur de Linux, EPERM pour Windows), et si l'URL ne correspond à aucun fichier, alors 404 not found.

Pour le fichier où je n'ai pas les droits de lecture, j'ai « dupliqué » le fichier bad.html en le renommant badSansLecture.html et en enlevant les droits de lecture. (j'ai dupliqué le fichier après son dépôt github)

```

C:\Users\flores>curl -4 http://localhost:1234/badSansLecture.html
<h1>403 Forbidden</h1>

```

- 7) Pour voir quelle est la méthode utilisée par le client, on rajoute `${req.method}` dans `console.log` (ligne 10)

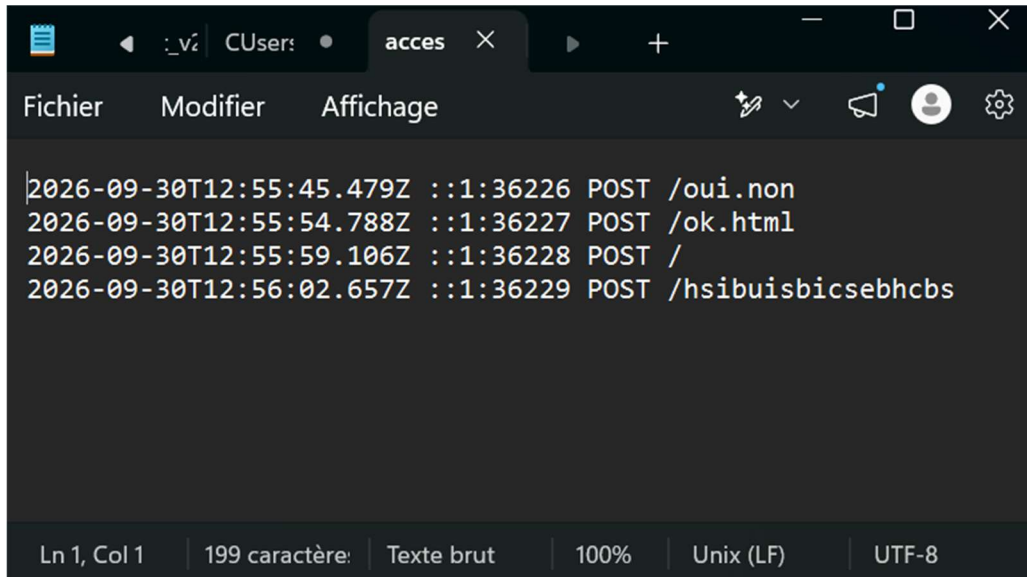
<pre> flore@LAPTOP-90B3947M UCRT64 ~\Desktop\R&T 2me nementielle/TD/R309_TD1_Florent_REUS-DORANDEU \$ node Question7 Serveur en écoute sur http://localhost:1234 Client ::1:13714 -> GET /ok.html Client ::1:13715 -> GET /oui.non </pre>	<pre> C:\Users\flores>curl http://localhost:1234/ok.html <!DOCTYPE html> <html> <head> <title>Page OK</title> </pre>
---	---

On remarque bien la méthode utilisée (GET)

8)

```
12   const ligne = `${new Date().toISOString()} ${remoteAddress}:${remotePort} ${req.method} ${req.url}\n`;
13   fs.appendFile(__dirname + '/access.log', ligne, (err) => {
14     if (err) console.log('Erreur d\'écriture du log :', err.message);
15   });
```

J'ai rajouté ce code, qui crée un fichier qui mentionne chaque connexion avec plusieurs informations, comme la date, l'adresse IP, la méthode et l'URL



```
2026-09-30T12:55:45.479Z ::1:36226 POST /oui.non
2026-09-30T12:55:54.788Z ::1:36227 POST /ok.html
2026-09-30T12:55:59.106Z ::1:36228 POST /
2026-09-30T12:56:02.657Z ::1:36229 POST /hsibuisbicsebhcb
```

On remarque toutes ces informations dans le fichier access.log